

Documentación Técnica de Proyecto: Sistema de Gestión de Usuarios con FastAPI, PostgreSQL, Prometheus y Grafana

Jose David Buitrago Aponte
Jose Emanuel Morcillo Sandoval

Facultad de Ingeniería
Universidad Autónoma de Occidente

Ingeniería de Datos e Inteligencia Artificial
Profesor: Heberth Martinez

Octubre de 2026

1. Definición del Proyecto

1.1 Identificación y caracterización del problema

En el desarrollo de software actual y en las plataformas de computación en la nube, los sistemas deben diseñarse bajo dos pilares indispensables: que puedan ejecutarse en cualquier entorno de manera confiable (contenerización) y que permitan conocer en todo momento su estado interno (observabilidad). En los proyectos tradicionales es muy común enfrentar problemas de reproducibilidad, conocidos coloquialmente como "en mi máquina sí funciona", ocasionados por discrepancias de versiones en Python, configuraciones dispersas de bases de datos o dependencias del sistema operativo anfitrión.

De igual forma, muchas aplicaciones operan como cajas negras: los equipos no tienen visibilidad de si un endpoint está respondiendo lento, cuántas personas se registran por minuto o si se generan errores no controlados. Este trabajo resuelve estas dificultades estructurando una solución integral en microservicios, donde cada componente asume una responsabilidad específica y todo el ecosistema se levanta coordinadamente con un solo comando.

1.2 Requerimientos de la solución

A partir de la guía académica, establecimos los requerimientos funcionales que determinan el alcance del sistema, detallados en la Tabla 1.

Tabla 1

Lista de requerimientos funcionales del sistema

Código	Nombre	Descripción
RF01	Operaciones CRUD	El sistema debe permitir crear, consultar, actualizar y eliminar usuarios mediante una API REST en FastAPI.
RF02	Persistencia	La información debe guardarse de forma permanente en una base de datos relacional PostgreSQL.
RF03	Métricas de Negocio	El sistema debe registrar y exponer métricas en tiempo real sobre la creación y gestión de usuarios.
RF04	Monitoreo en Tiempo Real	Los indicadores operativos y de rendimiento deben visualizarse dinámicamente mediante tableros en Grafana.

2. Diseño de la Arquitectura

2.1 Justificación de la contenerización y microservicios

Para erradicar instalaciones manuales y configuraciones propensas a fallos, separamos el sistema en contenedores independientes coordinados con Docker Compose. Cada servicio (la API web, la base de datos, el colector de telemetría y la plataforma visual) se ejecuta en un entorno aislado con sus dependencias resueltas de antemano.

Docker Compose unifica la infraestructura en un archivo declarativo (docker-compose.yml), habilitando una red interna privada donde los contenedores se descubren mediante nombres de host internos (DNS) y protegiendo los registros mediante volúmenes de datos con nombre para que la información perdure aunque los contenedores se detengan o reinicien.

2.2 Selección tecnológica

La elección de cada componente obedece a criterios de desempeño, robustez y estándares de la industria, tal como se resume en la Tabla 2.

Tabla 2
Selección de tecnologías por componente

Componente	Tecnología y Justificación
Backend	FastAPI (Python 3.11): Framework asíncrono moderno y veloz, con validación estricta de tipos mediante Pydantic y documentación OpenAPI interactiva en Swagger.
Base de Datos	PostgreSQL 13: Motor relacional robusto con transaccionalidad ACID, garantizando integridad en los registros frente a soluciones embebidas simples como SQLite.
Monitoreo	Prometheus: Sistema de recolección de series temporales pull-based (scrapping cada 5 segundos) que evita saturar la API y ofrece consultas flexibles en PromQL.
Visualización	Grafana: Plataforma líder de observabilidad analítica que permite construir paneles visuales y aprovisionar tableros mediante archivos de configuración en código.

2.3 Especificación de endpoints de la API REST

La interacción entre los usuarios y el backend se realiza mediante peticiones HTTP. La Tabla 3 describe los contratos y métodos implementados para la gestión de usuarios y la observabilidad.

Tabla 3
Especificación de los endpoints REST del backend

Método	Ruta	Propósito y Comportamiento
GET	/users	Lista todos los usuarios almacenados en la base de datos.
POST	/users	Registra un usuario nuevo en PostgreSQL e incrementa el contador user_created.
GET	/users/{id}	Consulta los datos de un usuario a partir de su identificador numérico.
PUT	/users/{id}	Modifica el nombre o la edad de un usuario registrado en el sistema.
DELETE	/users/{id}	Elimina físicamente el registro de un usuario en PostgreSQL.
GET	/metrics	Expone las métricas recolectadas para que Prometheus las lea periódicamente.
GET	/health	Comprueba que la base de datos esté activa respondiendo a una consulta de prueba.

3. Funcionamiento Integral del Sistema

3.1 Flujo operativo y comunicación

El ecosistema opera de manera coordinada a través de un ciclo continuo de procesamiento y telemetría:

[Cliente / Navegador / cURL]



[FastAPI en :8000] —(Persistencia SQL)—> [PostgreSQL en :5432]



(Scrape cada 5 segundos)



[Prometheus en :9090] —(Consulta PromQL)—> [Grafana en :3000]

1. **Recepción de solicitudes:** El usuario interactúa con la API desde el navegador (Swagger en el puerto 8000) o mediante terminal con cURL.
2. **Persistencia:** FastAPI procesa la petición mediante SQLAlchemy y aplica los cambios sobre PostgreSQL (puerto 5432).
3. **Conteo interno:** En cada creación o eliminación, FastAPI actualiza sus contadores en memoria y el middleware calcula la duración de la respuesta.
4. **Recolección (Scraping):** Prometheus realiza una petición HTTP cada 5 segundos al endpoint /metrics de la API y almacena las mediciones en su base de datos de series temporales.
5. **Renderizado visual:** Grafana consulta a Prometheus vía PromQL y actualiza en tiempo real los medidores, gráficos de líneas y contadores del tablero web (puerto 3000).

3.2 Mecanismo de arranque y persistencia

Para asegurar que los servicios arranquen sin condiciones de carrera, incorporamos dos salvaguardas técnicas:

- **Comprobación de salud (Healthcheck):** En Docker Compose, PostgreSQL evalúa periódicamente pg_isready. FastAPI no inicia hasta que PostgreSQL se reporte en estado healthy, evitando fallos de conexión al encender la máquina.
- **Reintentos con retroceso (Backoff):** En el código de FastAPI, la función init_db cuenta con un ciclo de hasta 15 intentos espaciados cada 2 segundos para asegurar la creación automática de las tablas de usuarios.
- **Persistencia en volúmenes:** Los registros de PostgreSQL y las series de Prometheus se almacenan en volúmenes nombrados de Docker (user_crud_postgres_data y user_crud_prometheus_data). De este modo, aunque se reinicien o apaguen los contenedores, los usuarios y las métricas permanecen intactos.

4. Monitoreo y Tableros en Grafana

4.1 Métricas implementadas

El backend instrumenta tanto métricas de negocio como métricas técnicas de tráfico HTTP:

- **user_created_total (Counter):** Cantidad total acumulada de usuarios creados en la API.
- **user_deleted_total (Counter):** Total de usuarios eliminados mediante el método DELETE.
- **users_total (Gauge):** Cantidad actual y viva de registros guardados en PostgreSQL.
- **http_requests_total (Counter):** Solicitudes recibidas clasificadas por endpoint, método y código HTTP.

- **http_request_duration_seconds (Histogram):** Medición de los tiempos de respuesta para calcular latencias.

4.2 Tablero preconfigurado en Grafana

Mediante aprovisionamiento automático (archivos en la carpeta grafana/provisioning), Grafana incluye la fuente de Prometheus y carga automáticamente el tablero FastAPI - CRUD & Performance Monitoring. La Tabla 4 describe los paneles que lo integran.

Tabla 4

Paneles del dashboard de monitoreo en Grafana

Panel	Tipo	Expresión PromQL	Descripción y Función
API Service Status	Stat	up{job="fastapi"}	Indica si el backend está activo (1 = Operativo, 0 = Inactivo).
Total Users Created	Stat	user_created_total or user_created	Contador acumulativo de altas de usuarios.
Current Users in DB	Stat	users_total	Muestra los usuarios que permanecen en la base de datos.
Users Deleted	Stat	user_deleted_total or vector(0)	Total de usuarios eliminados del sistema.
User Creation Rate	Time Series	rate(user_created_total[1m]) * 60	Tasa de creación medida en usuarios por minuto.
HTTP Throughput	Time Series	sum by (endpoint, method) (rate(http_requests_total[1m]))	Rendimiento de peticiones por segundo según el método.
HTTP Status Codes	Time Series	sum by (status_code)(http_requests_total)	Distribución de respuestas para supervisar códigos de error.
Latency p95	Time Series	histogram_quantile(0.95, sum(rate(...)))	Percentil 95 de tiempo de respuesta de la API en segundos.

5. Resumen de Implementación, Automatización y Pruebas

Toda la infraestructura se despliega con el comando `docker compose up --build -d`. Para facilitar la evaluación y generar tráfico de prueba en tiempo real, creamos el script `test_api.py`, el cual prueba ordenadamente la creación de usuarios, consultas, modificaciones y eliminaciones, permitiendo observar cómo suben los contadores y evolucionan las gráficas en Grafana de forma inmediata.

6. Conclusiones

El desarrollo de este proyecto demostró que la contenerización mediante Docker y Docker Compose simplifica radicalmente la entrega de software, permitiendo que cualquier evaluador ejecute una aplicación distribuida en segundos sin lidiar con configuraciones locales. Asimismo, integrar Prometheus y Grafana transforma una aplicación común en un sistema observable y profesional, brindando visibilidad en tiempo real sobre la salud del servicio y el comportamiento de los datos.